
Reproducing StraightPCF in Jittor: Point-Cloud Denoising, and Where the Reproduction Fell Short

Yi Hou

University of Chinese Academy of Sciences
Beijing, China
houyi25@mailsucas.ac.cn

Abstract

This report documents an attempt to reproduce StraightPCF (CVPR 2024), a point-cloud filtering method, in Jittor for Track 2 of the 6th Jittor AI Challenge. The final system is a coupled velocity module of roughly 0.7M parameters that predicts per-point displacements returning noisy samples to the underlying surface, trained on $\sim 20k$ ShapeNet meshes. On a held-out split of the training data it reduces the Chamfer distance by about 51% and the point-to-surface error by about 67% relative to the noisy input, giving a best competition score of 67.44/100 and a national top-100 placement. It does not reproduce the paper’s reported accuracy. Four causes account for most of the gap: a merged training schedule that trained the distance module differently from the paper’s prescription, noise-model drift away from the interpolation family the method assumes, inference-time repetition that compensated for a mediocre velocity field, and a score-based detour that consumed the final weeks. The implementation itself — chunked KNN, GPU farthest-point sampling, scatter-mean edge convolutions, MPI data parallelism, and an evaluation harness — is described in the middle sections.

1 Introduction

The 6th Jittor AI Challenge is a competition organized around Jittor, a JIT-compiled deep-learning framework developed at Tsinghua University. Track 2 asked entrants to denoise point clouds sampled from ShapeNet meshes: given a noisy cloud, predict a displacement for every point so that the displaced cloud lies close to the original surface. The task had to be solved in Jittor, the training data was a provided subset of $\sim 20k$ ShapeNet meshes, and submissions were scored on Chamfer distance (CD) and point-to-surface distance (P2S) relative to the noisy input. This report describes a solo entry: a reproduction of StraightPCF [1], a CVPR 2024 method that models filtering as an optimal-transport-like flow with a learned constant velocity field.

The system removes most of the noise and was accepted for scoring, but it does not reach the quality the paper reports, and it placed in the top 100 nationally. The reproduction also ran a training schedule that deviated from the paper’s; we diagnosed that deviation only after the submission window closed.

The report covers what was built and what went wrong. Section 2 defines the task, data, and metric. Section 3 describes the method as implemented, including the deviations from the paper. Section 4 is the systems core: the primitives that had to be written from scratch in Jittor and the memory discipline that Jittor’s eager-with-graph-execution model forced on us. Section 5 reports measured results. Section 6 is the post-mortem. Section 7 covers the qualification round. Sections 8–9 close with lessons and reproducibility.

2 Task, Data, and Metric

Task. The input is a noisy point cloud $\{p_i + n_i\}_{i=1}^N$ where the clean points lie on a shape surface normalized into the unit ball and the noise is Laplace-distributed with $\sigma \in [0.005, 0.020]$. The output is a point cloud with *exactly* the same number of points, float32, shape $(N, 3)$; points are expected to move back toward the surface, and the point count is checked by the organizers.

Data. Training uses approximately 20,000 ShapeNet meshes (.obj); the test set is a collection of pre-noised clouds (.npy). Because the test set ships without clean references, we built a self-evaluation harness (self_eval.py) that reserves a fraction of the training meshes (default 10%), samples points from each mesh, adds Laplace noise at the midpoint of the competition range, runs the full inference pipeline, and computes the competition metrics against the mesh. All numbers in Section 5 come from this harness.

Metric. Both metrics are means of *squared* nearest-neighbour distances. Chamfer distance is symmetric,

$$\text{CD}(A, B) = \frac{1}{|A|} \sum_{a \in A} \min_{b \in B} \|a - b\|_2^2 + \frac{1}{|B|} \sum_{b \in B} \min_{a \in A} \|b - a\|_2^2, \quad (1)$$

computed against the clean point set; P2S is one-directional and measured to the mesh surface (via a BVH, with a vertex-only fallback),

$$\text{P2S}(A, M) = \frac{1}{|A|} \sum_{a \in A} \min_{s \in M} \|a - s\|_2^2. \quad (2)$$

Both clouds are normalized to the unit sphere before measuring. The competition score maps each metric to a per-sample score and averages:

$$s_i = \text{clamp} \left(100 \left(1 - \frac{m_i^{\text{pred}}}{m_i^{\text{noisy}}} \right), 0, 100 \right), \quad \text{Score} = \frac{1}{2} \overline{s^{\text{CD}}} + \frac{1}{2} \overline{s^{\text{P2S}}}. \quad (3)$$

The score averages per-sample ratios, not the ratio of pooled means: a method can reduce pooled P2S by 67% and still show a P2S sub-score of 83, because samples where the prediction is far better than the noise pull the mean upward while a few hard samples are clamped at 0. We report both views in Section 5.

3 Method as Implemented

3.1 StraightPCF

StraightPCF [1] treats denoising as a transport problem: points are moved from the noisy distribution to the clean one along approximately straight paths, which makes single-step (or few-step) integration sufficient at inference. A velocity module v_θ is trained to predict a displacement that is *constant along the path* — for a pair (X_0, X_1) of high-noise and clean samples, the regression target is $X_1 - X_0$. Two velocity modules are coupled: VM1 moves the cloud one step, VM2 refines from the moved state, and a distance module d_ϕ predicts a scalar in $(0, 1)$ that scales the step size so that points stop *on* the surface rather than overshooting it. Inference is Euler integration with $T = \text{steps} \times K$ sub-steps, $K = 2$.

3.2 Data pipeline

Each training sample is built as follows. From a mesh we sample 32,768 surface points (plus 1,024 vertex samples used as seeds) and normalize the cloud to the unit ball, recording the center and scale for later denormalization. With probability 0.5 the cloud is rotated by a random Euler angle and rescaled by a factor in $[0.8, 1.2]$. We then cut 8 patches of 1,000 points each: a seed point is drawn from the (possibly noisy) cloud, its 1,000 nearest neighbours are taken from the *clean* cloud to form X_1 , a high-noise variant $X_0 = X_1 + \varepsilon_H$ is created with $\varepsilon_H \sim \mathcal{N}(0, \sigma_H^2)$ and $\sigma_H = 0.02$, and a per-point interpolation parameter $t \sim U(0, 1)$ gives the encoder input

$$X_t = (1 - t) X_0 + t X_1. \quad (4)$$

All three clouds are centered on the seed’s X_t position, which keeps displacements in a patch-local frame. Per-patch supervision uses 256 points in the competition configuration and all 1,000 points in the staged reruns.

Table 1: Architecture as implemented. Parameter counts are computed from the layer dimensions.

Component	Layers	Params
Feature extraction	EdgeConv 3→64, 64→128, 192→256 ($k = 16$)	260k
Decoder	Linear 256→256→64→3 + BN + dropout(0.1)	83k
Distance module	Linear 256→64→64→1, max over patch, sigmoid	21k
Velocity module (VM)	encoder + decoder	343k
Coupled model	VM1 + VM2 + distance module	707k

3.3 Architecture

Table 1 lists the network. The encoder is a three-layer dynamic EdgeConv stack: each layer builds a k -NN graph ($k = 16$) on its own input features and applies an MLP to the concatenation $[x_i, x_j - x_i]$ of a point and its edge vector, aggregates messages by scatter-mean, and adds a linear skip on x_i . The decoder is a small MLP with batch normalization and dropout. The distance module shares the encoder’s features, regresses a per-point scalar, reduces it by max over the patch, and applies a sigmoid (the paper’s Eq. 13 ordering). The coupled model is two velocity modules plus the distance module; parameters are counted in the table.

3.4 Training objectives

The velocity loss is the paper’s Eq. 7 with a $1/\sigma_{\text{dsm}}$ scaling ($\sigma_{\text{dsm}} = 0.01$),

$$\mathcal{L}_{\text{vel}} = \frac{1}{N} \sum_i \frac{1}{\sigma_{\text{dsm}}} \|v_{\theta}(X_t)_i - (X_1 - X_0)_i\|_2^2, \quad (5)$$

and the coupling term is the paper’s Eq. 10 with $\lambda_1 = 10$,

$$\mathcal{L}_{\text{cpl}} = \lambda_1 \|X_{t_1}^{\text{pred}} - X_{t_1}^{\text{ideal}}\|_2^2, \quad X_{t_1}^{\text{pred}} = X_t + \frac{1}{K} v_{\theta_1}(X_t), \quad t_1 = \frac{t(K-1)+1}{K}, \quad (6)$$

where $X_{t_1}^{\text{ideal}}$ is the interpolation point at t_1 . The distance module is trained with a direct term and a chain term,

$$\mathcal{L}_d = \|d_{\phi}(X_t) - (1-t)\|_2^2 + \lambda_2 \|\bar{X}_1 - X_1\|_2^2, \quad \lambda_2 = 200, \quad (7)$$

where $\bar{X}_1$ is the result of a short Euler chain (two steps through VM1/VM2) whose *only* differentiable input is d_{ϕ} ; this is what teaches d_{ϕ} to scale steps for convergence rather than merely regress $1-t$.

Training schedule. The paper trains in stages: pretrain VM1 alone, pretrain VM2 alone, couple them with $\mathcal{L}_{\text{vel}} + \mathcal{L}_{\text{cpl}}$ while the distance module is frozen, then train the distance module with the backbone frozen. Configurations for all four stages exist in the repository (configs/task/train_vm_stage{1..4}.yaml). The scored submission instead used a merged two-phase schedule: phase 1 trained both VMs with the coupling term and the distance module frozen; phase 2 trained the distance module briefly, then unfroze the backbone and continued training everything together. That is not the paper’s schedule and, as Section 6 argues, it is our central mistake. After the submission was frozen we reran the paper’s schedule: Stages 1 and 2 completed (Figure 1a), Stage 3 was started and abandoned, and Stage 4 was never run.

3.5 Inference

Inference is patch-based (Algorithm 1). Because encoding a full 32k-point cloud at once both mismatches the patch statistics seen in training and exceeds memory at useful batch sizes, we run the network on overlapping patches and blend the results. Seeds come from farthest-point sampling; each patch is the seed’s 1,000 nearest neighbours, centered on the seed. The Euler integrator runs three steps of the coupled model, giving $T = 3 \times 2 = 6$ sub-steps with step size d_{ϕ}/T . Patch outputs are combined by a soft assignment: a point’s weight for a patch is $\exp(-\tilde{d})$, where $\tilde{d}$ is the point’s KNN-rank distance inside the patch normalized by the patch radius; each point takes the prediction of the patch in which it is most central, and points assigned to no patch keep their noisy positions. We additionally repeat the whole patch-filtering pass 2–4 times based on an estimated noise level (mean 8-NN distance of a subsample) — a deviation from the paper’s single pass, discussed in Section 6.

Algorithm 1 Patch-based filtering of one noisy cloud

```
1: Input: noisy cloud  $P \in \mathbb{R}^{N \times 3}$ , patch size  $M=1000$ , seeds  $K \approx 6N/M$ 
2: Estimate noise level from mean 8-NN distance; set repeat count  $R \in \{2, 3, 4\}$ 
3: for  $r = 1$  to  $R$  do
4:    $\{s_k\} \leftarrow \text{FPS}(P, K)$  ▷ farthest-point seeds
5:    $\{P_k\} \leftarrow M\text{-NN patch of each } s_k$ ; center each patch on its seed
6:    $P'_k \leftarrow \text{Euler}_{\theta, \phi}(P_k)$  for all  $k$  (batched,  $\leq 8$  patches at a time)
7:   Re-center patches; blend:  $w_k(p) = \exp(-\tilde{d}_k(p))$ ;  $P \leftarrow$  per-point argmax assignment
8:   Points with no assignment keep their previous positions
9: end for
10: return  $P$  (denormalized)
```

4 Jittor Engineering

Jittor executes Python eagerly but builds and fuses the underlying computation graph with its own JIT, and it has no PyTorch-ecosystem counterpart for point-cloud primitives. Almost every geometric primitive this task needs — k -NN, farthest-point sampling, edge convolutions — had to be written by hand against Jittor ops, under memory behaviour quite different from PyTorch’s. This section documents those primitives and the memory discipline they required.

4.1 Primitives

Chunked k -NN. A direct broadcast $(N, 1) - (1, M)$ over a 1,000-point patch is fine, but k -NN is also needed against 32,768-point clouds, where the full distance matrix ($N \times M$ floats) is not. The implementation streams the reference cloud in chunks of 1,024, maintains a running top- k by concatenating each chunk’s distances with the running best and re-selecting, and records the original indices through gather. This keeps peak memory linear in the chunk size while remaining exact.

Farthest-point sampling. FPS is inherently sequential; our version does one argmax per selected point and thus one GPU→CPU synchronization per seed. That is acceptable when selecting a handful of patch seeds ($K \approx 6$ per 1,000 points) but was moved to a NumPy implementation for whole-cloud downsampling, where the seed count is large and the syncs dominated.

Edge convolution. Jittor has no scatter-mean kernel on GPU for this layout, so EdgeConv is implemented with dense `scatter_` (`reduce='add'`) over the destination index, followed by division by the accumulated count, plus the linear skip branch. This is exact and works on the (points, k) edge list produced by the graph builder.

NumPy/SciPy at the edges. Patch construction, patch blending, and the evaluation metrics use `scipy.spatial.cKDTree` on the CPU — not for speed but for predictability: KNN targets computed in the graph re-entered the computational graph and inflated memory, so a later variant (the score-based detour, Section 6) computes its targets entirely on the CPU.

4.2 Memory and the computation graph

Jittor accumulates the computation graph until it is executed and collected, and our first implementation fought both out-of-memory failures and intermittent segfaults. The crash signatures were misleading: one run died with a segfault after roughly 2,200 iterations, and a later failure read as out-of-memory although the device reported 45.9 GB of 47 GB in use with roughly 11,000 live operators and 12,000 live variables — the memory was held by the graph, not by the batch. The dominant cause was lazy graph accumulation: a single fusion kernel would hold several hundred k -NN and edge-convolution operators at once. The fix that made long runs possible was an explicit hygiene discipline: `jt.sync_all()` (execute and release) followed by `jt.gc()` after every encoder block, after every training step, and at epoch boundaries; a cap of 8 patches per inference batch; and detaching patch outputs from the graph before blending. The same discipline reduced the peak memory of an encoder forward pass enough to train at batch size 24 with 8 patches per sample. The syncs cost throughput, but the discipline turned a training run that reliably crashed into one that completed 100 epochs unattended.

Table 2: Headline results. Sub-scores and the final score follow the competition’s per-sample formula (Eq. 3); all figures are from the held-out self-evaluation split. The last row is a July rerun of the same architecture under the merged schedule.

Quantity	Noisy input	Best submission	July rerun
CD, pooled mean ($\times 10^{-4}$)	2.46	1.21	1.20
P2S, pooled mean ($\times 10^{-4}$)	1.96	0.64	0.72
CD sub-score	—	51.94	51.65
P2S sub-score	—	82.94	77.93
Final score	—	67.44	64.79

4.3 Vectorizing the reconstruction

The patch-blending reconstruction was initially written as per-patch Python loops, and because every comparison against the assignment mask crossed the GPU→CPU boundary via `.item()`, a single cloud’s reconstruction cost roughly 3×10^5 synchronizations. Rewriting the assignment with vectorized `jt.equal + jt.nonzero` masked scatter brought that to roughly 3×10^2 . The same pattern — find the Python loop that syncs per iteration, replace it with a masked tensor operation — recurred in FPS and in KNN.

4.4 Distributed training

Data-parallel training uses Jittor’s built-in MPI integration (`mpirun -np 6 python run.py ...`): parameters are broadcast at start via `mpi_param_broadcast`, validation losses are all-reduced, and checkpointing/logging are rank-0 only. Two practical notes: dataloaders are constructed before the CUDA context is initialized to avoid the fork-after-CUDA hazard, and the launch configuration carries `NCCL_P2P_DISABLE=1` as a workaround for peer-to-peer collective hangs on the shared node. Epoch time varied by configuration: 340–430 s for the submission run, about 936 s for the July reruns of the same coupled configuration, 1,400–1,900 s for the distance-module phase — its Euler chain adds four no-gradient encoder passes per step against two for the velocity modules alone — and about 813 s for Stage 1, which supervises all 1,000 points per patch.

4.5 Evaluation and profiling tooling

Three scripts shaped the work more than any model change. `self_eval.py` builds a held-out split and computes the full competition metric, including the mesh-based P2S, so that checkpoint selection could target the metric rather than the loss. `profile.py` attributes wall-clock time between data loading and GPU compute and auto-probes the batch size that fits. `vis_denoising.py` renders per-sample diagnostics — displacement direction error, magnitude, and error histograms — which is how we first saw that the network’s displacements were directionally decent but mis-scaled, the symptom that later pointed at the distance module.

5 Results

5.1 Denoising quality

Table 2 gives the headline numbers for the best scored submission. The best score was 67.44/100, with the Chamfer half (51.9) noticeably weaker than the P2S half (82.9). Pooled over samples, CD falls from 2.46×10^{-4} to 1.21×10^{-4} (a 51% reduction) and P2S from 1.96×10^{-4} to 6.4×10^{-5} (a 67% reduction). The per-sample P2S sub-score (82.9) is higher than the pooled reduction suggests because the score average rewards samples where the prediction beats the noise by a wide margin while clamping the few bad samples at zero.

5.2 Training dynamics

Two observations from Figure 1 shaped our reading of the whole effort. The staged pretraining runs are reproducible: VM1 and VM2, trained independently with the same recipe from different seeds, follow nearly identical trajectories ($3.78 \rightarrow 0.101$ over 15 epochs), so the training machinery itself is

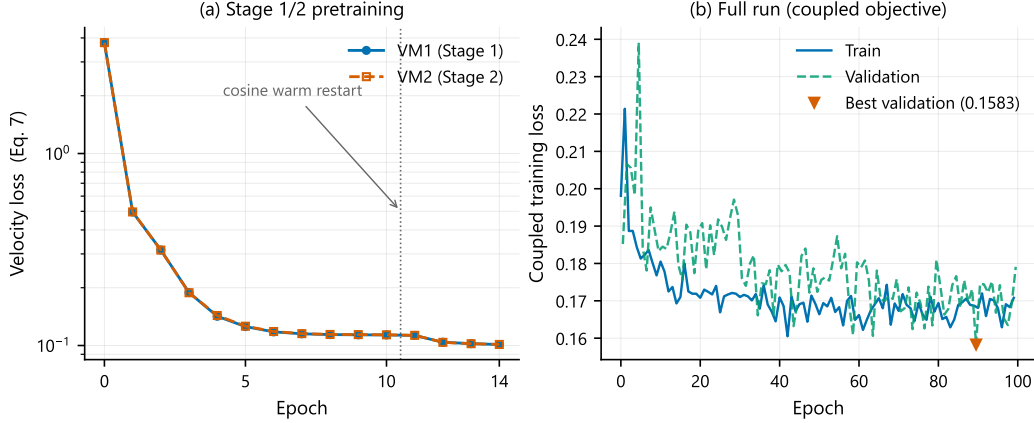


Figure 1: (a) Stage 1/2 pretraining (Eq. 7 velocity loss, one value per epoch) from the staged rerun of July 2026. The two independent runs are nearly indistinguishable, and the loss falls three orders of magnitude before the cosine warm restart at epoch 11; both settle at ≈ 0.101 . (b) The full coupled training run of May 2026 that produced the best submission (train and validation loss, one epoch per point). Both curves plateau by epoch ~ 40 ; the remaining 60 epochs did not improve the metric.

stable. Both the pretraining and the coupled run also reach their loss floors quickly — within 5 and about 40 epochs respectively — after which hundreds of additional epochs changed neither the loss nor the metric. We treated the plateau as an optimization problem for too long. The loss is a proxy for the metric, and Section 6 argues the binding constraint was the training target rather than the optimizer. The staged rerun that followed stopped before producing a usable model: run alone on the competition’s noise distribution, a pretrained single velocity module scores 1.01/100 — essentially no improvement over the noisy input (CD sub-score 0.00, P2S 2.03).

5.3 What changed the numbers

The changes we can attribute improvements to, ordered by confidence: (i) cutting 8 patches of 1,000 points per sample instead of one (an $8\times$ increase in supervised points per sample); (ii) moving inference from whole-cloud Euler integration to patch-based filtering, which fixed both a train/inference statistics mismatch and the memory ceiling; (iii) feeding the encoder the same interpolation state that training uses when computing the coupled state (X_{t_1}), removing a small distribution shift; (iv) removing the distance module from the phase-1 forward pass (encoder calls per step $3 \rightarrow 2$); and (v) the reconstruction vectorization and sync discipline of Sections 4. Items (i)–(iii) were validated by the competition metric; (iv)–(v) are throughput and stability wins.

6 Post-Mortem: What Failed and Why

The merged schedule and the distance module. The paper’s staged schedule is not incidental: the distance module is trained last *with the backbone frozen*, so that it learns to scale step sizes against a stationary feature distribution. Our competition schedule instead trained the distance module briefly and then unfroze the backbone, after which the features it had learned against changed. The July rerun quantifies the damage: with the backbone unfrozen and the distance module along for the ride, the P2S sub-score fell from 82.9 to 77.9 while CD was unchanged. This matches a step-size regulator that no longer stops points on the surface, with the direction of motion (which CD is more sensitive to) unaffected. It is our best explanation for the gap to the paper, and we found it after the submission window had closed.

Noise-model drift. The method’s mathematics assumes interpolation between a noisy and a clean sample of the *same* noise family: the velocity target $X_1 - X_0$ is meaningful because X_t moves along that line. The competition’s noise is Laplace. We trained with a “mixed” schedule (60% Laplace, 20% Gaussian, 20% uniform) intended to hedge, which quietly violates the interpolation assumption for two-fifths of the samples. The hedge bought nothing measurable and cost fidelity to the method’s

premise; the staged reruns use pure Gaussian per the paper. The 1.01/100 measurement of Section 5 makes the cost concrete: a velocity module trained only on Gaussian noise does not transfer to Laplace noise at all. We should have matched the evaluation noise distribution exactly rather than hedging inside a framework whose math assumes a specific family.

Inference repetition. Our inference repeats the patch filter 2–4 times depending on estimated noise. This raises the score — iterated Euclidean projections onto an imperfectly straight field behave like a curved path — but the reported quality then partly reflects inference compute rather than the velocity field. Under the paper’s single pass we could not reproduce the paper’s numbers.

A plateau we did not diagnose in time. Loss floors at epoch 5 (pretraining) and ~ 40 (coupled) with no metric movement afterward should have redirected us from learning-rate tuning to the objective itself: the loss is a proxy, and nothing guaranteed that driving it to its floor would optimize CD/P2S. The metric harness that could have told us this earlier was itself built mid-competition — a sequencing mistake.

The score-based detour. In the final weeks we implemented a ScoreDenoise-style [2] variant reusing the same encoder/decoder: the target becomes the empirical score $\text{NN}(X_t, X_{\text{clean}}) - X_t$ and inference becomes annealed Langevin dynamics. It is distribution-agnostic — it assumes neither a noise family nor straight paths — which is the property our Laplace/Gaussian mismatch needed. The implementation hit memory problems on the $O(N^2)$ nearest-neighbour target over 1,000-point patches, solved first by chunking and finally by computing targets on the CPU with `cKDTree`; the code exists in the repository (`src/model/score_vm.py`) and was never trained to a comparable state before the deadline. It consumed the last stretch of the competition and produced no scored result.

Process. Three process failures, in decreasing order of cost: the staged reproduction (the correct move) started after the submission was frozen, not before it; the official-metric harness arrived late, so weeks were spent optimizing a proxy loss; and with a single part-time participant, the last-mile work — running the proper schedule to completion — simply did not fit the window.

7 The Qualification Round: PCT for ModelNet40

The track opened with a qualification round: classify ModelNet40 shapes from 2,048 points into 40 categories, in Jittor, with a test-accuracy threshold of 80% to advance. Our entry ranked 12th in the round.

7.1 Model

The final model follows the full architecture of the PCT paper [3] ($\sim 2.9\text{M}$ parameters): a convolutional stem ($3 \rightarrow 64 \rightarrow 64$), two sample-and-group stages (farthest-point sampling $1024 \rightarrow 512 \rightarrow 256$, k -NN with $k = 32$, local feature aggregation), four offset-attention layers with position encoding and residual connections, a concatenated skip into a $1280 \rightarrow 1024$ projection, and a max-pooled classifier ($1024 \rightarrow 512 \rightarrow 256 \rightarrow 40$).

Two details deviate from the reference implementation. The four attention layers each carry their own positional projection rather than sharing one; we have no ablation that isolates the effect of this change. And the low-level operators are hand-written: `rf_ops.py` implements farthest-point sampling with a shared-memory reduction, a ported k -NN, and index-based point gathering as Jittor CUDA ops.

7.2 Training recipe, and what actually helped

AdamW (lr 0.01, weight decay 10^{-4}), warmup-cosine schedule (10-epoch warmup, $\eta_{\min} = 10^{-6}$), batch size 32, 300 epochs, label smoothing 0.1, dropout 0.5, and weight EMA. Augmentations: random scaling $[0.67, 1.5]$, translation ± 0.2 , point dropout 10–40% (with resampling to preserve the point count), and Gaussian jitter ($\sigma = 0.01$, clipped). Rotation augmentation was present in earlier versions and removed during tuning; that is the only augmentation change with code evidence behind it. Validation used a fixed 90/10 split.

The version history is short. A minimal PCT — four attention layers, no sample-and-group stages, rotation-only augmentation, SGD with cosine annealing — already cleared the threshold at roughly 80% accuracy. An experiment that added a geometric attention bias (a learnable rescaling of pairwise Euclidean distances) together with LayerNorm and a feed-forward block underperformed and was dropped. The gains that mattered came from the hierarchical sample-and-group stages and the widened augmentation set. Per-version accuracies were not retained; the 12th-place finish is the only measured result that survives.

7.3 A Jittor gotcha worth recording

Jittor’s dropout layers did not follow the module’s training flag reliably in our version. The final training script works around this by walking the module tree and setting every dropout layer explicitly (`_set_dropout_train`) at the start of each epoch and before validation. Until that fix, runs trained with dropout in an unintended state. This is a silent regularization change, not a crash, which is what makes it expensive: nothing in the loss curves or the logs announces it.

The main track drew the opposite lesson from this round. Here, the data-side work — augmentation ranges, the sample-and-group stages, the validation split — produced the gains, and architecture variants did not. On the main track, weeks went into architecture and schedule variants while the noise model and the evaluation harness were fixed late.

8 Lessons Learned

- *The training schedule is part of the method.* The largest gap to the paper came from a training-order deviation we chose for expedience; when components are trained against each other’s frozen states, the order determines what they learn.
- *A loss floor is not a metric result.* Both losses reached clean floors and the metric did not follow. The held-out harness for the real metric belonged in the first week.
- *Hedging inside a method’s assumptions does not pay.* Mixed-noise training cost fidelity to the interpolation premise and bought nothing measurable.
- *Inference tricks that compensate hide what is broken.* Repeating the filter raised the score and concealed a velocity field that was not straight enough.
- *Memory discipline is part of the model in Jittor.* `sync_all/gc` placement, chunked KNN, and patch-batch caps made long runs possible and change what can be trained, not just how fast.
- *Look for per-iteration `.item()` calls first.* They dominated reconstruction until vectorized; the pattern repeated in FPS and KNN.
- *MPI data parallelism was cheap; the primitives were not.* Jittor’s broadcast/all-reduce integration worked in an afternoon, and the time went to KNN, memory, and the metric.

9 Reproducibility

The repository contains everything needed to rebuild and rerun the pipeline. Training and inference run through `denoise/run.py` with per-task YAML configs under `denoise/configs/`: `train_vm.yaml` (the merged schedule that produced the submission), `train_vm_stage{1..4}.yaml` (the paper’s schedule), `predict_vm.yaml`, and `train_score.yaml` (the untrained score-based variant). Evaluation is `self_eval.py` (held-out split, full metric) and `evaluate.py` (official-style scoring); diagnostics are `profile.py` and `vis_denoising.py`. Figures and tables in this report are generated by `report/make_figures.py` from the run logs (the May run log is included under `report/data/`). The dataset — ShapeNet meshes and the pre-noised test clouds — is provided by the competition organizers and is not redistributed here; datalist files define the splits.

Acknowledgement of scope. The task definition, dataset, and evaluation protocol belong to the organizers of the 6th Jittor AI Challenge (Track 2). The StraightPCF method and its equations are due to de Silva Edirimuni et al. [1]. The implementation in Jittor, the experiments, the evaluation harness, and all measurements and errors in this report are my own.

References

- [1] Dasith de Silva Edirimuni, Xuequan Lu, Gang Li, Lei Wei, Antonio Robles-Kelly, and Hongdong Li. StraightPCF: Straight point cloud filtering. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2024.
- [2] Shitong Luo and Wei Hu. Score-based point cloud denoising. In *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*, 2021.
- [3] Meng-Hao Guo, Jun-Xiong Cai, Zheng-Ning Liu, Tai-Jiang Mu, Ralph R. Martin, and Shi-Min Hu. PCT: Point cloud transformer. *Computational Visual Media*, 7(2):187–199, 2021.
- [4] Charles R. Qi, Hao Su, Kaichun Mo, and Leonidas J. Guibas. PointNet: Deep learning on point sets for 3D classification and segmentation. In *CVPR*, 2017.